

# Real-Time Examples of Single Responsibility Principle (SRP) in C#

In this article, I will discuss the **Multiple Real-Time Examples of the Single Responsibility Principle (SRP) in C#**. Please read our previous article discussing the basic concepts of the [Single Responsibility Principle in C#](#). At the end of this article, you will understand the following Real-time Examples using the Single Responsibility Principle in C#.

1. [Banking System](#)
2. [Education System](#)
3. [Report Generation System](#)
4. [Management of Books in a Library System](#)
5. [Managing Employee Salaries](#)
6. [Handling Customer Orders](#)
7. [Vehicle Maintenance System](#)
8. [Event Management System](#)
9. [Online Article Publishing](#)
10. [Managing Hotel Room Bookings](#)

**Note:** First, I will show the example without following SRP. Then, I will discuss the problem of not following the Single Responsibility Principle, and then we will write the same example following the Single Responsibility Principle.

## Real-Time Example of Single Responsibility Principle in C#: Banking System

Let's understand the real-time example of a Banking System using the Single Responsibility Principle. A banking system allows customers to maintain a bank account where they can deposit and withdraw money. Additionally, the bank wants to provide a facility to print a statement of the account's transaction history.

### Violation of SRP:

The BankAccount class might combine transaction handling and statement printing in a simple design. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```
using System;
using System.Collections.Generic;
namespace SRPDemo
{
    public class BankAccount
    {
        public int AccountNumber { get; private set; }
        public double Balance { get; private set; }
        private List<string> Transactions = new List<string>();
        public BankAccount(int accountNumber)
        {
            AccountNumber = accountNumber;
        }
        public void Deposit(double amount)
```

```

    {
        Balance += amount;
        Transactions.Add($"Deposited ${amount}. New Balance: ${Balance}");
    }
    public void Withdraw(double amount)
    {
        Balance -= amount;
        Transactions.Add($"Withdrew ${amount}. New Balance: ${Balance}");
    }
    public void PrintStatement()
    {
        Console.WriteLine($"Statement for Account: {AccountNumber}");
        foreach (var transaction in Transactions)
        {
            Console.WriteLine(transaction);
        }
    }
}

```

Here, the BankAccount class handles:

- Transaction operations.
- Printing the transaction statement.

### Following SRP:

- A cleaner approach would separate transaction management from statement printing:
- BankAccount manages the transactions of the account.
- StatementPrinter handles printing the transaction statement.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```

using System;
using System.Collections.Generic;
namespace SRPDemo
{
    public class BankAccount
    {
        public int AccountNumber { get; private set; }
        public double Balance { get; private set; }
        public List<string> Transactions = new List<string>();
        public BankAccount(int accountNumber)
        {
            AccountNumber = accountNumber;
        }
        public void Deposit(double amount)
        {
            Balance += amount;

```

```

        Transactions.Add($"Deposited ${amount}. New Balance: ${Balance}");
    }
    public void Withdraw(double amount)
    {
        Balance -= amount;
        Transactions.Add($"Withdrew ${amount}. New Balance: ${Balance}");
    }
}
public class StatementPrinter
{
    public void Print(BankAccount account)
    {
        Console.WriteLine($"Statement for Account: {account.AccountNumber}");
        foreach (var transaction in account.Transactions)
        {
            Console.WriteLine(transaction);
        }
    }
}

//Testing the Single Responsibility Principle
public class Program
{
    public static void Main()
    {
        BankAccount johnsAccount = new BankAccount(123456);
        johnsAccount.Deposit(500);
        johnsAccount.Withdraw(100);
        StatementPrinter printer = new StatementPrinter();
        printer.Print(johnsAccount);

        Console.ReadKey();
    }
}

```

In this design, BankAccount focuses solely on transaction-related operations. Only this class is affected if there's a change in how transactions are managed. Conversely, StatementPrinter focuses on printing the statement. If the format or medium of the statement needs a change, only the StatementPrinter class requires modifications. This adherence to SRP makes the system modular and maintainable. When you run the above code, you will get the following output.

```

Statement for Account: 123456
Deposited $500. New Balance: $500
Withdrew $100. New Balance: $400

```

## Real-Time Example of Single Responsibility Principle in C#: Education System

Let's see another real-time example Education System using the Single Responsibility Principle. In a university's digital system, students are enrolled in courses, and at the end of a semester, they receive grades for each course. The university wants to calculate each student's GPA (Grade Point Average) based on their grades. Additionally, the system should generate a transcript showing all courses, grades, and the calculated GPA.

### Violation of SRP:

In an elementary design, the Student class might handle course enrollment, grade assignment, GPA calculation, and transcript generation. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace SRPDemo
{
    public class Student
    {
        public string Name { get; set; }
        private Dictionary<string, double> CoursesAndGrades = new Dictionary<
double>();
        public void EnrollCourse(string courseName)
        {
            CoursesAndGrades[courseName] = 0; // default grade
        }
        public void AssignGrade(string courseName, double grade)
        {
            if (CoursesAndGrades.ContainsKey(courseName))
            {
                CoursesAndGrades[courseName] = grade;
            }
        }
        public double CalculateGPA()
        {
            // Basic GPA calculation logic
            return CoursesAndGrades.Values.Average();
        }
        public void PrintTranscript()
        {
            Console.WriteLine($"Transcript for {Name}");
            foreach (var course in CoursesAndGrades)
            {
                Console.WriteLine($"{course.Key}: {course.Value}");
            }
            Console.WriteLine($"GPA: {CalculateGPA()}");
        }
    }
}
```

Here, the Student class has multiple responsibilities:

- Managing course enrollments.
- Handling grade assignments.
- Calculating GPA.
- Printing the transcript.

### Following SRP:

A clearer approach would segregate these functionalities:

- The student manages course enrollments and grade assignments.
- GPACalculator calculates the GPA for a student.
- TranscriptGenerator creates and prints the student's transcript.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace SRPDemo
{
    public class Student
    {
        public string Name { get; set; }
        public Dictionary<string, double> CoursesAndGrades = new Dictionary<string, double>();
        public void EnrollCourse(string courseName)
        {
            CoursesAndGrades[courseName] = 0; // default grade
        }
        public void AssignGrade(string courseName, double grade)
        {
            if (CoursesAndGrades.ContainsKey(courseName))
            {
                CoursesAndGrades[courseName] = grade;
            }
        }
    }
    public class GPACalculator
    {
        public double CalculateGPA(Student student)
        {
            // Basic GPA calculation logic
            return student.CoursesAndGrades.Values.Average();
        }
    }
    public class TranscriptGenerator
    {
        private GPACalculator _gpaCalculator;
        public TranscriptGenerator(GPACalculator gpaCalculator)
```

```

    {
        _gpaCalculator = gpaCalculator;
    }
    public void PrintTranscript(Student student)
    {
        Console.WriteLine($"Transcript for {student.Name}");
        foreach (var course in student.CoursesAndGrades)
        {
            Console.WriteLine($"{course.Key}: {course.Value}");
        }
        Console.WriteLine($"GPA: {_gpaCalculator.CalculateGPA(student)}");
    }
}

//Testing the Single Responsibility Principle
public class Program
{
    public static void Main()
    {
        Student alice = new Student { Name = "Alice" };
        alice.EnrollCourse("Mathematics");
        alice.AssignGrade("Mathematics", 90);
        GPACalculator gpaCalc = new GPACalculator();
        TranscriptGenerator transcriptGen = new TranscriptGenerator(gpaCalc);
        transcriptGen.PrintTranscript(alice);

        Console.ReadKey();
    }
}

```

By adhering to SRP, each class has a clear, singular duty. If the university decides to change the GPA calculation method, only the GPACalculator requires modification. Similarly, if the transcript format needs adjustments, only the TranscriptGenerator is affected. This modular design simplifies future changes and maintenance.

## Real-Time Example of Single Responsibility Principle in C#: Report Generation System

Let's use the real-time example of a report generation system to illustrate the Single Responsibility Principle (SRP) in C#. Suppose a company wants to generate employee reports detailing their monthly hours and pay. The reports can be printed or saved as a PDF.

### Violation of SRP:

A simple design might combine the responsibility of generating the report's content with the responsibility of rendering it in different formats. Let us see how we can implement the above example without following the Single Responsibility Principle in C#

```
using System;
```

```

namespace SRPDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public int HoursWorked { get; set; }
        public decimal Pay { get; set; }
    }
    public class EmployeeReport
    {
        private Employee _employee;
        public EmployeeReport(Employee employee)
        {
            _employee = employee;
        }
        public string GenerateReportContent()
        {
            return $"Employee: {_employee.Name}, Hours: {_employee.HoursWorked}, Pay: {_employee.Pay}";
        }
        public void Print()
        {
            // Print logic here using GenerateReportContent
        }
        public void SaveAsPDF()
        {
            // PDF saving logic here using GenerateReportContent
        }
    }
}

```

Here, the EmployeeReport class is responsible for generating the report content and managing how that content is rendered (printed or saved as a PDF).

### Following SRP:

A better design would separate the responsibility of generating content from rendering it:

- EmployeeReport is responsible for generating the report content.
- ReportPrinter is responsible for printing the report.
- ReportPDFSaver is responsible for saving the report as a PDF.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```

using System;
namespace SRPDemo
{
    public class Employee
    {
        public string Name { get; set; }
    }
}

```

```

        public int HoursWorked { get; set; }
        public decimal Pay { get; set; }
    }
    public class EmployeeReport
    {
        private Employee _employee;
        public EmployeeReport(Employee employee)
        {
            _employee = employee;
        }
        public string GenerateReportContent()
        {
            return $"Employee: {_employee.Name}, Hours: {_employee.HoursWorked}, Pay: {_employee.Pay}";
        }
    }
    public class ReportPrinter
    {
        public void Print(string content)
        {
            // Print logic here
            Console.WriteLine($"Report Printed: {content}");
        }
    }
    public class ReportPDFSaver
    {
        public void SaveAsPDF(string content, string filePath)
        {
            // PDF saving logic here
            Console.WriteLine($"Report Saved as PDF, content: {content} and Path: {filePath}");
        }
    }
    //Testing the Single Responsibility Principle
    public class Program
    {
        public static void Main()
        {
            Employee john = new Employee { Name = "John", HoursWorked = 160, Pay = 1600 };

            EmployeeReport report = new EmployeeReport(john);
            string content = report.GenerateReportContent();
            ReportPrinter printer = new ReportPrinter();
            printer.Print(content);
            ReportPDFSaver pdfSaver = new ReportPDFSaver();
            pdfSaver.SaveAsPDF(content, "path_to_save_report.pdf");
            Console.ReadKey();
        }
    }
}

```

Now, each class has a single responsibility. If there's a need to change how content is generated, only the EmployeeReport class is affected. If there's a need to modify how content is printed or saved, only the respective classes (ReportPrinter or ReportPDFSaver) need to change. This design adheres to SRP, making the system more maintainable and extendable.

## Real-Time Example of Single Responsibility Principle in C#: Management of Books in a Library

Let's consider a real-world example of the management of books in a library. A library wants to manage its books using a system. They want to add new books, delete old ones, and display details about a book. Furthermore, they want the ability to log the changes they make.

### Violation of SRP:

A simple approach might combine the management of book details and logging actions into a single class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```
namespace SRPDemo
{
    public class LibraryBook
    {
        public string Title { get; set; }
        public string Author { get; set; }
        public int ISBN { get; set; }
        public void AddBook()
        {
            // Logic to add the book to the library's system
            Log($"Added {Title} by {Author}");
        }
        public void DeleteBook()
        {
            // Logic to delete the book from the library's system
            Log($"Deleted {Title} by {Author}");
        }
        public void DisplayDetails()
        {
            // Logic to display the details of the book
        }
        private void Log(string message)
        {
            // Logic to log changes into a log file
        }
    }
}
```

In this design, the LibraryBook class has multiple responsibilities:

- Managing book details.

- Logging actions.

### Following SRP:

A better approach would separate the book management from the logging responsibility:

- LibraryBook is responsible for managing the book's details.
- LibraryLogger is responsible for logging library actions.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```
using System;
namespace SRPDemo
{
    public class LibraryBook
    {
        public string Title { get; set; }
        public string Author { get; set; }
        public int ISBN { get; set; }
        public void AddBook()
        {
            // Logic to add the book to the library's system
            Console.WriteLine("Book Added to the Library");
        }
        public void DeleteBook()
        {
            // Logic to delete the book from the library's system
            Console.WriteLine("Book Deleted from the Library");
        }
        public void DisplayDetails()
        {
            // Logic to display the details of the book
            Console.WriteLine("Displaying the Book Details");
        }
    }
    public class LibraryLogger
    {
        public void Log(string message)
        {
            // Logic to log changes into a log file
            Console.WriteLine(message);
        }
    }

    //Testing the Single Responsibility Principle
    public class Program
    {
        public static void Main()
        {

```

```

        LibraryBook book = new LibraryBook
        {
            Title = "SOLID Principle",
            Author = "Pranaya Rout",
            ISBN = 123456789
        };
        LibraryLogger logger = new LibraryLogger();
        book.AddBook();
        logger.Log($"Added {book.Title} by {book.Author}");
        book.DeleteBook();
        logger.Log($"Deleted {book.Title} by {book.Author}");
        Console.ReadKey();
    }
}
}

```

With this design, the responsibility of managing book details is isolated in the LibraryBook class, while the logging actions are managed by the LibraryLogger class. This ensures that if there's a change needed in logging mechanisms (e.g., from file logging to database logging), only the LibraryLogger class needs modifications, adhering to the Single Responsibility Principle. When you run the above code, you will get the following output.

```

Book Added to the Library
Added SOLID Principle by Pranaya Rout
Book Deleted from the Library
Deleted SOLID Principle by Pranaya Rout

```

## Real-Time Example of Single Responsibility Principle in C#: Managing Employee Salaries

Let's consider another real-time example to illustrate the Single Responsibility Principle (SRP) in C#: Managing Employee Salaries. A company wants to calculate salaries for its employees. Each employee has a basic salary, but bonuses can be added based on performance. The company also wants the ability to save the salary details in a database.

### Violation of SRP:

If we combine the responsibility of calculating the salary and saving it to a database into one class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```

namespace SRPDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public double BasicSalary { get; set; }
        public double Bonus { get; set; }
        public double CalculateTotalSalary()
    }
}

```

```

        {
            return BasicSalary + Bonus;
        }
        public void SaveSalaryToDatabase()
        {
            // Logic to save salary details to a database
        }
    }
}

```

In the design above, the Employee class has the responsibilities of:

- Managing and calculating salary details.
- Handling database operations related to salary.

### Following SRP:

A more suitable design would separate the calculation of salary from the responsibility of saving it to a database:

- EmployeeSalary is responsible for managing and calculating the salary.
- SalaryDatabaseHandler is responsible for handling database operations related to salary.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```

using System;
namespace SRPDemo
{
    public class EmployeeSalary
    {
        public string EmployeeName { get; set; }
        public double BasicSalary { get; set; }
        public double Bonus { get; set; }
        public double CalculateTotalSalary()
        {
            return BasicSalary + Bonus;
        }
    }
    public class SalaryDatabaseHandler
    {
        public void SaveSalaryDetails(EmployeeSalary salary, double totalSalary)
        {
            // Logic to save salary details to a database
            Console.WriteLine($"Employee {salary.EmployeeName} Salary {totalSalary} saved to DB");
        }
    }
}

//Testing the Single Responsibility Principle

```

```

public class Program
{
    public static void Main()
    {
        EmployeeSalary johnSalary = new EmployeeSalary
        {
            EmployeeName = "Pranaya",
            BasicSalary = 5000,
            Bonus = 500
        };
        double totalSalary = johnSalary.CalculateTotalSalary();
        SalaryDatabaseHandler dbHandler = new SalaryDatabaseHandler();
        dbHandler.SaveSalaryDetails(johnSalary, totalSalary);

        Console.ReadKey();
    }
}

```

In this design, the EmployeeSalary class only worries about salary-related functionalities. If there's a need to change how the salary is calculated, this class can be modified without affecting database-related operations.

On the other hand, SalaryDatabaseHandler handles the database operations. If there's a need to change how salary details are stored in the database or if there's a need to switch to a new database system, only this class would require changes.

This separation of concerns, where each class has only one responsibility, makes the system clearer, more modular, and easier to maintain.

## Real-Time Example of Single Responsibility Principle in C#: Handling Customer Orders

Let's take another Real-time example to understand the Single Responsibility Principle, i.e., handling customer orders in an e-commerce system. An e-commerce platform needs to manage customer orders. After a customer places an order, the system should calculate the total order amount, generate an order receipt, and notify the customer via email.

### Violation of SRP:

A basic design might bundle all these operations into a single class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```

using System.Collections.Generic;
using System.Linq;
namespace SRPDemo
{
    public class Order
    {
        public List<OrderItem> Items { get; private set; } = new List<OrderItem>

```

```

    public Customer Customer { get; set; }
    public double CalculateOrderAmount()
    {
        return Items.Sum(item => item.Price);
    }
    public string GenerateReceipt()
    {
        // Logic to generate a receipt for the order
        return $"Receipt for {Customer.Name}: Total - {CalculateOrderAmount}";
    }
    public void NotifyCustomer()
    {
        // Send an email to the customer with the order details
    }
}
public class OrderItem
{
    public string ProductName { get; set; }
    public double Price { get; set; }
}
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }
}
}

```

Here, the Order class has multiple responsibilities:

- Calculating the order amount.
- Generating a receipt.
- Notifying the customer.

### Following SRP:

A better approach would split these responsibilities into separate classes:

- Order handles the order's basic functionalities.
- ReceiptGenerator handles receipt generation.
- CustomerNotifier handles customer notifications.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace SRPDemo
{
    public class OrderItem

```

```

{
    public string ProductName { get; set; }
    public double Price { get; set; }
}
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }
}
public class Order
{
    public List<OrderItem> Items { get; set; } = new List<OrderItem>();
    public Customer Customer { get; set; }
    public double CalculateOrderAmount()
    {
        return Items.Sum(item => item.Price);
    }
}
public class ReceiptGenerator
{
    public string GenerateReceipt(Order order)
    {
        // Logic to generate a receipt based on the given order
        return $"Receipt for {order.Customer.Name}: Total -
{order.CalculateOrderAmount()}";
    }
}
public class CustomerNotifier
{
    public void NotifyCustomer(Order order)
    {
        // Logic to notify the customer about their order
        Console.WriteLine("Notification Sent to Customer");
    }
}

//Testing the Single Responsibility Principle
public class Program
{
    public static void Main()
    {
        Order newOrder = new Order();
        // Add items to the order
        List<OrderItem> Items = new List<OrderItem>()
        {
            new OrderItem(){ProductName = "Laptop", Price = 10000},
            new OrderItem(){ProductName = "Desktop", Price = 7000},
            new OrderItem(){ProductName = "Tab", Price = 3000}
        };
        newOrder.Items = Items;
        //set the customer...
    }
}

```

```

        Customer customer = new Customer()
        {
            Name = "Pranaya",
            Email = "Pranaya@dotnettutorials.net"
        };
        newOrder.Customer = customer;
        ReceiptGenerator receiptGen = new ReceiptGenerator();
        string receipt = receiptGen.GenerateReceipt(newOrder);
        Console.WriteLine(receipt);
        CustomerNotifier notifier = new CustomerNotifier();
        notifier.NotifyCustomer(newOrder);

        Console.ReadKey();
    }
}

```

In this approach, each class has a single responsibility. If the receipt format needs to be changed, only ReceiptGenerator needs modification. If the notification mechanism needs an update (e.g., incorporating SMS notifications), only CustomerNotifier requires changes. This decoupling makes the system more flexible, maintainable, and less prone to errors. When you run the above code, you will get the following output.

```

Receipt for Pranaya: Total - 20000
Notification Sent to Customer

```

## Real-Time Example of Single Responsibility Principle in C#: Vehicle Maintenance System

Let's consider another real-world scenario of a Vehicle Maintenance System. A system is designed to manage vehicles. The system can log maintenance details for a vehicle and provide alerts for scheduled services.

### Violation of SRP:

A simple design might encapsulate the management of vehicle details, maintenance logging, and service alerts into a single class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```

using System;
namespace SRPDemo
{
    public class Vehicle
    {
        public string VIN { get; set; }
        public string Model { get; set; }
        public DateTime LastServiceDate { get; set; }
        public void LogMaintenance(string details)
        {
            // Logic to log maintenance details

```

```

    }
    public bool IsServiceDue()
    {
        // Logic to determine if the vehicle is due for service
        return (DateTime.Now - LastServiceDate).TotalDays > 365;
    }
    public void SendServiceAlert()
    {
        // Logic to send a service alert to the vehicle owner
    }
}
}

```

Here, the Vehicle class has multiple responsibilities:

- Managing vehicle details.
- Logging maintenance records.
- Handling service alerts.

### Following SRP:

A cleaner approach separates the vehicle's management from its maintenance and service alerts:

- Vehicle manages the basic details of the vehicle.
- MaintenanceLog manages logging and retrieving maintenance details.
- ServiceAlertHandler manages service alerts.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```

using System;
namespace SRPDemo
{
    public class Vehicle
    {
        public string VIN { get; set; }
        public string Model { get; set; }
        public DateTime LastServiceDate { get; set; }
    }
    public class MaintenanceLog
    {
        private Vehicle _vehicle;
        public MaintenanceLog(Vehicle vehicle)
        {
            _vehicle = vehicle;
        }
        public void LogMaintenance(string details)
        {
            // Logic to log maintenance details for the given vehicle
            Console.WriteLine(details);
        }
    }
}

```

```

    }
}
public class ServiceAlertHandler
{
    private Vehicle _vehicle;
    public ServiceAlertHandler(Vehicle vehicle)
    {
        _vehicle = vehicle;
    }
    public bool IsServiceDue()
    {
        // Logic to determine if the vehicle is due for service
        return (DateTime.Now - _vehicle.LastServiceDate).TotalDays > 365;
    }
    public void SendServiceAlert()
    {
        if (IsServiceDue())
        {
            // Logic to send a service alert to the vehicle owner
            Console.WriteLine("Service Alert Send to Customer");
        }
    }
}

//Testing the Single Responsibility Principle
public class Program
{
    public static void Main()
    {
        Vehicle myCar = new Vehicle { VIN = "XYZ123", Model = "Model S",
LastServiceDate = DateTime.Now.AddMonths(-13) };
        MaintenanceLog maintenance = new MaintenanceLog(myCar);
        maintenance.LogMaintenance("Replaced brake pads.");
        ServiceAlertHandler alertHandler = new ServiceAlertHandler(myCar);
        if (alertHandler.IsServiceDue())
        {
            alertHandler.SendServiceAlert();
        }

        Console.ReadKey();
    }
}
}

```

Now, each class has a clear, singular responsibility. Maintenance details are separate from the vehicle's core attributes, and service alerts are decoupled from both. This makes it easier to maintain, modify, or extend functionalities independently. When you run the above code, you will get the following output.

```

Replaced brake pads.
Service Alert Send to Customer

```

## Real-Time Example of Single Responsibility Principle in C#: Event Management

Let's consider another real-world example of an Event Management System. An event management platform is designed to create and promote events. An organizer can create an event, save its details, and share the event details on social media.

### Violation of SRP:

A simple approach might integrate the functionalities of event creation, storage, and sharing into a single class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```
using System;
namespace SRPDemo
{
    public class Event
    {
        public string Name { get; set; }
        public DateTime Date { get; set; }
        public string Location { get; set; }
        public void SaveEvent()
        {
            // Logic to save the event to a database or file
        }
        public void ShareOnSocialMedia()
        {
            // Logic to share the event details on various social media platf
        }
    }
}
```

In the class above, Event has multiple responsibilities:

- Representing the event details.
- Saving the event to storage.
- Sharing the event on social media.

### Following SRP:

A more structured approach would disentangle these functionalities:

- Event class holds the core event details.
- EventStorage handles saving and retrieving event details.
- EventPromoter manages the promotion of events on different platforms.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```
using System;
```

```

namespace SRPDemo
{
    public class Event
    {
        public string Name { get; set; }
        public DateTime Date { get; set; }
        public string Location { get; set; }
    }

    public class EventStorage
    {
        public void SaveEvent(Event eventDetail)
        {
            // Logic to save the event to a database or file
            Console.WriteLine("Event Saved to the Database");
        }

        public Event GetEvent(string eventName)
        {
            // Logic to retrieve the event based on its name
            Console.WriteLine($"Fetching the Event {eventName} Details");

            return new Event(); // Placeholder return for demonstration
        }
    }

    public class EventPromoter
    {
        public void ShareOnSocialMedia(Event eventDetail)
        {
            // Logic to share the event details on various social media platforms
            Console.WriteLine("Event Details Shared on Multiple Social Media Platforms");
        }
    }

    //Testing the Single Responsibility Principle
    public class Program
    {
        public static void Main()
        {
            Event concert = new Event { Name = "Rock Concert", Date =
DateTime.Now.AddDays(30), Location = "City Arena" };
            EventStorage storage = new EventStorage();
            storage.SaveEvent(concert);
            EventPromoter promoter = new EventPromoter();
            promoter.ShareOnSocialMedia(concert);

            Console.ReadKey();
        }
    }
}

```

Each class upholds a singular responsibility with this design, making it straightforward and modular. If you need to adjust how events are stored (say, from a file-based system to a database), only the EventStorage

class needs modification. Similarly, if the social media sharing logic needs updates or additional platforms to be integrated, the EventPromoter class would be the focal point of changes without affecting the core event representation or storage mechanism. When you run the above code, you will get the following output.

```
Event Saved to the Database
Event Details Shared on Multiple Social Media Platforms
```

## Real-Time Example of Single Responsibility Principle in C#: Online Article Publishing

Let's understand another real-world example of an Online Article Publishing System. A website allows authors to write articles. Once written, the articles can be saved to a database. An overview of the article (like a summary) can also be generated for preview purposes.

### Violation of SRP:

A rudimentary approach might combine the creation, storage, and summarization functionalities into one class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```
namespace SRPDemo
{
    public class Article
    {
        public string Title { get; set; }
        public string Content { get; set; }
        public void SaveToDatabase()
        {
            // Logic to save the article to a database
        }
        public string GenerateSummary()
        {
            // Logic to generate a summary from the content
            return Content.Substring(0, 100) + "..."; // just a simple example
        }
    }
}
```

In this design, the Article class has multiple duties:

- Representing the article.
- Managing database storage.
- Creating a content summary.

### Following SRP:

A clearer design would disassociate these responsibilities:

- Article class is responsible for holding article information.
- ArticleDatabaseManager manages saving and retrieving articles.
- ArticleSummarizer generates summaries.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```
using System;
namespace SRPDemo
{
    public class Article
    {
        public string Title { get; set; }
        public string Content { get; set; }
    }
    public class ArticleDatabaseManager
    {
        public void SaveArticle(Article article)
        {
            // Logic to save the article to the database
        }
        public Article GetArticle(string title)
        {
            // Logic to retrieve an article from the database by its title
            return new Article(); // Placeholder for the demonstration
        }
    }
    public class ArticleSummarizer
    {
        public string GenerateSummary(Article article)
        {
            // Logic to generate a summary from the article content
            return article.Content.Substring(0, 100) + "..."; // just a simple
        }
    }

    //Testing the Single Responsibility Principle
    public class Program
    {
        public static void Main()
        {
            Article newArticle = new Article
            {
                Title = "The Benefits of SRP",
                Content = "A long content about the Single Responsibility Principle"
            };
            ArticleDatabaseManager dbManager = new ArticleDatabaseManager();
            dbManager.SaveArticle(newArticle);
            ArticleSummarizer summarizer = new ArticleSummarizer();
            string summary = summarizer.GenerateSummary(newArticle);
        }
    }
}
```

```

    Console.ReadKey();
}
}
}

```

With this design, each class is aligned with a specific responsibility. If the method to save articles to the database needs to change, you'd only adjust the `ArticleDatabaseManager`. If there's a new strategy to summarize articles, only `ArticleSummarizer` requires modification. This separation aligns with SRP and leads to a more maintainable and extensible codebase.

## Real-Time Example of Single Responsibility Principle in C#: Managing Hotel Room Bookings

Let's understand another real-time example, i.e., Managing Hotel Room Bookings. A hotel wants a system where customers can book rooms. The system should be able to book a room, check its availability, and generate a booking confirmation invoice.

### Violation of SRP:

A simple design might merge booking functionalities, availability checks, and invoice generation into one class. Let us see how we can implement the above example without following the Single Responsibility Principle in C#:

```

namespace SRPDemo
{
    public class HotelRoom
    {
        public int RoomNumber { get; set; }
        public bool IsBooked { get; private set; }
        public void BookRoom()
        {
            if (!IsBooked)
            {
                IsBooked = true;
                // Other booking logic
            }
        }
        public bool CheckAvailability()
        {
            return !IsBooked;
        }
        public string GenerateInvoice()
        {
            if (IsBooked)
            {
                // Logic to generate invoice for the booking
                return $"Invoice for Room {RoomNumber}";
            }
            return string.Empty;
        }
    }
}

```

```
}  
}
```

In the above design, the HotelRoom class handles:

- Room booking.
- Availability checking.
- Invoice generation.

### Following SRP:

A more organized approach would separate these responsibilities:

- HotelRoom represents the room's details and its booking status.
- RoomBookingManager handles the room booking process.
- InvoiceGenerator takes care of generating booking invoices.

Let us see how we can implement the above example following the Single Responsibility Principle in C#:

```
using System;  
namespace SRPDemo  
{  
    public class HotelRoom  
    {  
        public int RoomNumber { get; set; }  
        public bool IsBooked { get; private set; }  
        public void MarkAsBooked()  
        {  
            IsBooked = true;  
        }  
    }  
    public class RoomBookingManager  
    {  
        public bool BookRoom(HotelRoom room)  
        {  
            if (!room.IsBooked)  
            {  
                room.MarkAsBooked();  
                return true;  
            }  
            return false;  
        }  
        public bool CheckAvailability(HotelRoom room)  
        {  
            return !room.IsBooked;  
        }  
    }  
    public class InvoiceGenerator  
    {
```

```

    public string GenerateInvoice(HotelRoom room)
    {
        if (room.IsBooked)
        {
            // Logic to generate an invoice for the booked room
            return $"Invoice for Room {room.RoomNumber}";
        }
        return string.Empty;
    }
}

//Testing the Single Responsibility Principle
public class Program
{
    public static void Main()
    {
        HotelRoom room101 = new HotelRoom { RoomNumber = 101 };
        RoomBookingManager bookingManager = new RoomBookingManager();
        bool isBooked = bookingManager.BookRoom(room101);
        if (isBooked)
        {
            InvoiceGenerator invoiceGen = new InvoiceGenerator();
            string invoice = invoiceGen.GenerateInvoice(room101);
        }

        Console.ReadKey();
    }
}

```

With this design, each class adheres to a single responsibility. If the booking logic changes, only the RoomBookingManager requires modifications. If there's a new format or added details for invoices, you'd only adjust the InvoiceGenerator. This approach makes the system more maintainable and modular and adheres to the Single Responsibility Principle.

In the next article, I will discuss the [Open-Closed Principle in C#](#) with Examples. In this article, I discussed multiple **Real-Time Examples of the Single Responsibility Principle (SRP) in C#**. I hope you enjoy this Single Responsibility Principle (SRP) Real-Time Examples using the C# article.